

DSL DE TESTES UNITÁRIOS PARA LINGUAGEM FUNCIONAL 3 (LF3)

Equipe: Ericlecio Thiago e Anderson M. Marinho

O PROBLEMA

- Fail-Fast: O interpretador para no primeiro erro.
- Impossibilidade de testar o "Caminho Triste" (cenários de falha).
- Regressão: Dificuldade de validar novas regras sem quebrar as antigas.

```
let
  var sacar = fn saldo valor . if saldo > valor then saldo - valor else saldo
in
  (sacar(100, 20) == 80 ) ..

  sacar(10, 50) ..

  (sacar(200, 50) == 150 ) // Esse teste final NUNCA será executado
```

O PROBLEMA

- Fail-Fast: O interpretador para no primeiro erro.
- Impossibilidade de testar o "Caminho Triste" (cenários de falha).
- Regressão: Dificuldade de validar novas regras sem quebrar as antigas.

```
let
  var sacar = fn saldo valor . if saldo > valor then saldo - valor else saldo
in
  (sacar(100, 20) == 80 ) ..

  sacar(10, 50) ..

  (sacar(200, 50) == 150 ) // Esse teste final NUNCA será executado
```

A SOLUÇÃO

- DSL Nativa na AST inspirada no mercado (Jest, HUnit).
- Padrão BDD (Behavior-Driven Development).
- Execução Isolada (Non-blocking).

GRAMÁTICA (BNF)

- ExpDescribe ::= "describe" "(" Expressao "," ListExp ")"
- ExpTest ::= "test" "(" Expressao "," Expressao ")"
- ExpExpect ::= "expect" "(" Expressao "," Expressao ")"
- ExpToThrow ::= "toThrow" "(" Expressao ")"

ENGENHARIA DA AST (CLASSES JAVA) Funcional3.jj

```
TOKEN : /* TOKENS DE FUNCIONAL 3 */
```

```
{  
  < AND : "and" >  
  < OR : "or" >  
  < NOT : "not" >  
  < LENGTH : "length" >  
  < TRUE : "true" >  
  < FALSE : "false" >  
  < LET : "let" >  
  < VAR : "var" >  
  < IN : "in" >  
  < IF : "if" >  
  < THEN : "then" >  
  < ELSE : "else" >  
  < FUNC : "fun" >  
  < FN : "fn" >  
  < HEAD : "head" >  
  < TAIL : "tail" >  
  < FOR : "for" >  
}
```

```
/* --- NOVOS TOKENS DA DSL DE TESTES --- */
```

```
{  
  < DESCRIBE : "describe" >  
  < TEST : "test" >  
  < EXPECT : "expect" >  
  < TOTHROW : "toThrow" >  
}
```

```
/* --- REGRAS DA DSL DE TESTES --- */
```

```
Expressao PExpExpect() :
```

```
{  
  Expressao expEsq, expDir;  
}  
{  
  < EXPECT > < LPAREN > expEsq = PExpressao() < COMMA > expDir = PExpressao() <  
  {  
    return new ExpExpect(expEsq, expDir);  
  }  
}
```

```
Expressao PExpToThrow() :
```

```
{  
  Expressao exp;  
}  
{  
  < TOTHROW > < LPAREN > exp = PExpressao() < RPAREN >  
  {  
    return new ExpToThrow(exp);  
  }  
}
```

```
Expressao PExpTest() :
```

```
{  
  Expressao descricao;  
  Expressao assercao;  
}  
{  
  < TEST > < LPAREN > descricao = PExpressao() < COMMA > assercao = PExpressao()  
  {  
    return new ExpTest(descricao, assercao);  
  }  
}
```

```
Expressao PExpPrimaria() :
```

```
{  
  Expressao retorno;  
}  
{  
  // Removidos LOOKAHEADs manuais excessivos para d  
  (  
    retorno = PExpMenos()  
    | retorno = PExpNot()  
    | retorno = PExpLength()  
    | retorno = PExpHead()  
    | retorno = PExpTail()  
    | retorno = PExpDeclaracao()  
    | retorno = PExpCondicional()  
    | retorno = PExpCompreensaoLista()  
    | retorno = PExpDescribe()  
    | retorno = PExpTest()  
    | retorno = PExpExpect()  
    | retorno = PExpToThrow()  
    | retorno = PExpPrimaria()  
  )  
  {  
    return retorno;  
  }  
}
```

- **FUNÇÃO:** ATUA COMO UM CONTAINER DE TESTES (AGRUPADOR LÓGICO).
- **ARMAZENAMENTO:** RECEBE UMA DESCRIÇÃO E UMA LISTA DE EXPTEST.
- **GERENCIAMENTO:** O MÉTODO AVALIAR() GERENCIA O CICLO DE VIDA: LIMPA O ESTADO, EXECUTA OS TESTES E RETORNA O RESULTADO.

[GERENCIADORDETESTES] --> [EXPDESCRIBE
(GERENCIAMENTO)] --> [EXPTTEST (EXECUÇÃO)]

```
@Override
public Valor avaliar(AmbienteExecucao amb) throws VariavelNaoDeclaradaException, VariavelJaDeclaradaException {
    String nomeSuite = ((ValorString) descricao.avaliar(amb)).valor();
    GerenciadorDeTestes manager = GerenciadorDeTestes.getInstancia();

    manager.reset();

    for (Expressao teste : testes) {
        teste.avaliar(amb);
    }

    String relatorio = manager.gerarRelatorio(nomeSuite);
    System.out.println(relatorio);
    manager.exportarResultadosJson(nomeSuite);

    return new ValorString(relatorio);
}

@Override
public Expressao reduzir(AmbienteExecucao amb) throws VariavelNaoDeclaradaException, VariavelJaDeclaradaException {
    return avaliar(amb);
}

@Override
public boolean checaTipo(AmbienteCompilacao amb)
    throws VariavelNaoDeclaradaException, VariavelJaDeclaradaException {
    return true;
}
```

ENGENHARIA DA AST (CLASSES JAVA) ExpTest

- **RESPONSABILIDADE:** EXECUTA UM CENÁRIO DE TESTE INDIVIDUAL.
- **RESILIÊNCIA:** USO DE TRY-CATCH(THROWABLE) PARA PREVENIR O ENCERRAMENTO DO INTERPRETADOR.
- **INTEGRAÇÃO:** REGISTRA O SUCESSO OU FALHA NO GERENCIADORDETESTES.
- **GERENCIADOR DE COMPILAÇÃO:** O MÉTODO CHECATIPO() RETORNA TRUE PARA PERMITIR VALIDAÇÕES DINÂMICAS.

```
@Override
public Valor avaliar(AmbienteExecucao amb) throws VariavelNaoDeclaradaException, VariavelJaDeclaradaException {
    GerenciadorDeTestes manager = GerenciadorDeTestes.getInstancia();
    String nomeDoTeste = null;
    try {
        Valor descValor = descricao.avaliar(amb);
        if (!(descValor instanceof ValorString)) {
            throw new TesteFalhouException(mensagem: "A descrição do teste deve ser uma string.");
        }
        nomeDoTeste = ((ValorString) descValor).valor();
        assercao.avaliar(amb);
        manager.registrarSucesso();
        return new ValorBooleano(valor: true);
    } catch (TesteFalhouException e) {
        manager.registrarFalha(nomeRotulo(nomeDoTeste), mensagemOuTipo(e));
        return new ValorBooleano(valor: false);
    } catch (Throwable t) {
        manager.registrarFalha(nomeRotulo(nomeDoTeste), mensagemErroNativo(t));
        return new ValorBooleano(valor: false);
    }
}
```

```
@Override
public boolean checaTipo(AmbienteCompilacao amb)
    throws VariavelNaoDeclaradaException, VariavelJaDeclaradaException {
    try {
        System.out.println("Debug: " + this.getClass().getSimpleName() + " checado com sucesso.");
        return true;
    } catch (Exception e) {
        System.out.println("Debug: ERRO em " + this.getClass().getSimpleName() + ": " + e.getMessage());
        return false;
    }
}
```

ENGENHARIA DA AST (CLASSES JAVA) ExpExpect

- **RESPONSABILIDADE:** IMPLEMENTA A ASSERÇÃO LÓGICA (COMPARAÇÃO ENTRE VALOR REAL E ESPERADO).
- **COMPARAÇÃO:** UTILIZA O MÉTODO EQUALS() POLIMÓRFICO DOS VALORES DA LF3.
- **INTEGRAÇÃO:** LANÇA TESTEFALHOUEXCEPTION PARA SINALIZAR DIVERGÊNCIAS AO EXPTEST.

```
@Override
public Valor avaliar(AmbienteExecucao amb) throws VariavelNaoDeclaradaException, VariavelJaDeclaradaExcept
    Valor valorEsq = expressaoEsquerda.avaliar(amb);
    Valor valorDir = expressaoDireita.avaliar(amb);

    if (valorEsq.equals(valorDir)) {
        return new ValorBooleano(valor: true);
    } else {
        throw new TesteFalhouException("Esperado " + valorDir + ", mas obteve " + valorEsq);
    }
}

@Override
public Expressao reduzir(AmbienteExecucao amb) throws VariavelNaoDeclaradaException, VariavelJaDeclaradaE
    return avaliar(amb);
}
```

ENGENHARIA DA AST (CLASSES JAVA) ExpToThrow

- **OBJETIVO:** VALIDAR O "CAMINHO TRISTE" (CENÁRIOS DE FALHA ESPERADA).
- **LÓGICA DE INVERSÃO:** O TESTE É BEM-SUCEDIDO QUANDO UMA EXCEÇÃO É CAPTURADA, NÃO QUANDO A EXPRESSÃO É AVALIADA COM SUCESSO.
- **SEGURANÇA:** GARANTE QUE O INTERPRETADOR BLOQUEIE OPERAÇÕES INVÁLIDAS CONFORME A ESPECIFICAÇÃO.

```
@Override
public Valor avaliar(AmbienteExecucao amb) throws VariavelNaoDeclaradaException, VariavelJaDeclaradaException {
    try {
        expressaoAlvo.avaliar(amb);
        throw new TesteFalhouException(
            mensagem: "Era esperada uma exceção nativa, mas a expressão foi avaliada com sucesso.");
    } catch (TesteFalhouException e) {
        throw e;
    } catch (Throwable e) {
        return new ValorBooleano(valor: true);
    }
}

@Override
public Expressao reduzir(AmbienteExecucao amb) throws VariavelNaoDeclaradaException, VariavelJaDeclaradaException {
    return avaliar(amb);
}

@Override
public boolean checaTipo(AmbienteCompilacao amb)
    throws VariavelNaoDeclaradaException, VariavelJaDeclaradaException {
    return true;
}
```

ENGENHARIA DA AST (CLASSES JAVA) TesteFalhouException

- **PROPÓSITO:** SINALIZAÇÃO SEMÂNTICA DE FALHAS EM ASSERÇÕES.
- **SEMÂNTICA:** DIFERENCIA UMA "FALHA DE TESTE" (ESPERADA) DE UM "ERRO FATAL DO INTERPRETADOR" (INESPERADO).
- **IMPLEMENTAÇÃO:** ESTENDE RUNTIMEEXCEPTION PARA FACILITAR A PROPAGAÇÃO NA ÁRVORE DE EXECUÇÃO DA AST.

```
lf3 > plp > functional3 > exception > TesteFalhouException.java > { } lf3.plp.functional3.ex
package lf3.plp.functional3.exception;
public class TesteFalhouException extends RuntimeException {
    public TesteFalhouException(String mensagem) {
        super(mensagem);
    }
}
```

ENGENHARIA DA AST (CLASSES JAVA) GerenciadorDeTestes

- **PADRÃO SINGLETON:** GARANTE ACESSO ÚNICO E CENTRALIZADO AO ESTADO DOS TESTES.
- **INTEGRIDADE DOS DADOS:** ARMAZENA CONTADORES (PASSARAM/FALHARAM) E LOGS DETALHADOS.
- **SAÍDA ESTRUTURADA:** GERA RELATÓRIOS EM TEXTO E EXPORTAÇÃO AUTOMÁTICA EM JSON.

```
public void registrarSucesso() {
    testesPassaram++;
}

public void registrarFalha(String nomeTeste, String motivo) {
    testesFalharam++;
    logsDeFalha.add("[FALHOU] " + nomeTeste + " -> " + motivo);
}

public String gerarRelatorio(String nomeSuite) {
    StringBuilder sb = new StringBuilder();
    sb.append("\n=== RELATÓRIO DE TESTES: ").append(nomeSuite).append(" ===\n");
    sb.append("Total Executados: ").append(testesPassaram + testesFalharam).append("\n");
    sb.append("Passaram: ").append(testesPassaram).append("\n");
    sb.append("Falharam: ").append(testesFalharam).append("\n");

    if (testesFalharam > 0) {
        sb.append("\nDetalhes das Falhas:\n");
        for (String log : logsDeFalha) {
            sb.append(log).append("\n");
        }
    }

    sb.append("=====\n");
    return sb.toString();
}
```

EXEMPLO PRÁTICO (BDD)



**OBRIGADO PELA
ATENÇÃO!**